
Plan de Pruebas

para

Tarjeta Más Juventud

Versión 1.0 aprobada

Preparada por: Daniel Mongeote Tlachy

H. Ayuntamiento de Xalapa

21/05/2026

Tabla de contenido

1.	Introducción.....	4
1.1.	Antecedentes	4
1.2.	Visión y Alcance	4
1.3.	Supuestos y restricciones	5
1.4.	Roles y responsabilidades	5
1.5.	Capacitación	6
2.	Ambiente de pruebas y herramientas requeridas.....	7
2.1.	Ambientes de pruebas	7
2.2.	Diagrama del Ambiente de desarrollo.....	8
2.3.	Herramientas de soporte.....	9
3.	Estrategia general de las pruebas.....	10
3.1.	Fases de prueba	10
3.1.1.	Planeación de prueba	10
3.1.2.	Definición de pruebas	11
3.1.3.	Ejecución de pruebas	11
3.2.	Estrategia de pruebas.....	12
3.2.1.	Criterios de entrada, salida y suspensión de pruebas	12
3.2.2.	Trazabilidad de las pruebas.....	13
3.2.3.	Problemas de las pruebas.....	13
3.2.4.	Principios de las pruebas.....	14
4.	Pruebas unitarias.....	15
4.1.	Objetivos	16
4.1.1.	Alcance	17
4.1.2.	Casos de prueba	18
4.1.3.	Fuentes de datos.....	23
4.1.4.	Resultados de las pruebas	25
5.	Gestión de incidencias.....	28
6.	Resultados y métricas	30
6.1.	Casos ejecutados	30
6.2.	Casos aprobados.....	31
6.3.	Casos fallidos	31

6.4. Incidencias.....	32
Apéndice A: Evidencias.....	33

Índice de tablas

Tabla 1. Roles del plan de pruebas y responsabilidades asociadas.....	6
Tabla 2. Necesidades de capacitación.....	6
Tabla 3. Ambientes de prueba	8
Tabla 4. Herramientas de soporte	9
Tabla 5. Casos de prueba para las pruebas unitarias.....	20
Tabla 6. pruebas unitarias de autenticación	20
Tabla 7. pruebas unitarias de almacenamiento	21
Tabla 8. pruebas unitarias de obtención de datos	21
Tabla 9. pruebas unitarias de paginación.....	22
Tabla 10. pruebas unitarias del dashboard.....	22
Tabla 11. pruebas unitarias de caché api	22
Tabla 12. pruebas unitarias del login	23
Tabla 13. Fuentes de datos para las pruebas unitarias (valor)	24
Tabla 14. fuente de datos para las pruebas unitarias (archivo)	25
Tabla 15. Gestión de incidencias	29
Tabla 16. Distribución general de las pruebas ejecutadas	30
Tabla 17. resultado general de aprobación	31
Tabla 18. estado final de los casos fallidos.....	32
Tabla 19. métricas generales de las incidencias.....	33

Control de cambios

Nombre	Fecha	Motivo del cambio	Versión

1. Introducción

1.1. Antecedentes

En los últimos años, la administración pública ha experimentado una transformación significativa impulsada por la adopción de tecnologías de la información, dando origen al concepto de Gobierno Electrónico. Este enfoque busca optimizar la prestación de servicios gubernamentales, mejorar la transparencia y fomentar una mayor cercanía con la ciudadanía mediante el uso de plataformas digitales.

En el municipio de Xalapa, el H. Ayuntamiento ha promovido iniciativas orientadas a la modernización de sus procesos administrativos, integrando soluciones tecnológicas que permitan facilitar el acceso a servicios y programas públicos. Dentro de este contexto, la Dirección de Juventud ha impulsado programas dirigidos a la población joven, especialmente aquellos enfocados en el acceso a beneficios económicos, descuentos y oportunidades de desarrollo.

Sin embargo, estos programas tradicionalmente han operado de manera dispersa y con bajo nivel de digitalización, lo que ha limitado su alcance, dificultado su difusión y reducido la participación de los jóvenes beneficiarios. Asimismo, la falta de herramientas tecnológicas ha impedido establecer una vinculación efectiva entre los comercios locales y la población joven, así como medir el impacto real de estas iniciativas en el ámbito económico y social.

Ante esta situación, surge la necesidad de implementar una solución tecnológica integral que permita centralizar, digitalizar y optimizar el acceso a dichos beneficios. En respuesta a esta problemática, se propone el desarrollo del proyecto “Tarjeta Más Juventud”, el cual busca aprovechar las tecnologías móviles para mejorar la interacción entre ciudadanos, comercios y el gobierno municipal, contribuyendo así al fortalecimiento del ecosistema digital y al desarrollo social de la juventud en Xalapa.

1.2. Visión y Alcance

El presente Plan de Pruebas tiene como propósito definir la estrategia de validación que será aplicada al sistema web “Tarjeta Más Juventud”, estableciendo las actividades, recursos, criterios y responsabilidades necesarias para verificar el correcto funcionamiento de sus principales componentes durante el desarrollo del proyecto.

El alcance de este plan se limita a la ejecución de pruebas unitarias, pruebas de integración y pruebas de usabilidad, debido a que estas permiten evaluar los aspectos esenciales del sistema en su versión inicial. Las pruebas unitarias permitirán comprobar el funcionamiento individual de los componentes desarrollados, tales como formularios, validaciones, servicios, funciones auxiliares y reglas de negocio específicas. Las pruebas de integración permitirán validar la comunicación entre los distintos módulos del sistema, incluyendo la interacción entre la interfaz web, los servicios del *backend* y la base de datos. Finalmente, las pruebas de usabilidad permitirán evaluar si la plataforma resulta clara, comprensible y fácil de utilizar para los usuarios principales, especialmente jóvenes beneficiarios y personal administrativo.

Dentro del alcance se contempla la validación de funcionalidades como el registro y autenticación de usuarios, modificación de cuenta, administración de empresas, registro de sucursales, gestión de promociones, consulta de empresas, visualización de *dashboards* y operación general de los flujos definidos en los casos de uso del sistema.

Quedan fuera del alcance de este plan las pruebas de rendimiento, seguridad, regresión, conversión de datos, *smoke test*, accesibilidad formal y pruebas de aceptación de usuario independientes, ya que no serán ejecutadas como parte de esta etapa del proyecto. No obstante, algunos aspectos básicos relacionados con seguridad, rendimiento o accesibilidad podrán ser observados de manera complementaria durante las pruebas definidas, sin considerarse objetivos formales del presente plan.

1.3. Supuestos y restricciones

La correcta ejecución del presente Plan de Pruebas parte de una serie de supuestos operativos, técnicos y organizacionales que permitirán llevar a cabo las actividades definidas. En primer lugar, se asume la disponibilidad de ambientes tecnológicos adecuados para desarrollo, pruebas y preproducción, así como acceso controlado a la infraestructura necesaria, incluyendo servidor web, base de datos y red institucional.

De igual forma, se considera que el equipo responsable del proyecto contará con los recursos humanos mínimos requeridos para atender actividades de análisis, desarrollo, pruebas, corrección de incidencias y validación funcional. Asimismo, se asume la participación de usuarios clave pertenecientes a la Dirección de Juventud para la ejecución de pruebas de aceptación y validación de procesos reales del negocio.

En cuanto a herramientas, se contempla la disponibilidad de software de apoyo para pruebas funcionales, seguimiento de incidencias, validación de *APIs*, consultas a base de datos y monitoreo básico de desempeño.

Entre las principales restricciones identificadas se encuentran los tiempos limitados derivados del calendario institucional, la posible disponibilidad parcial del personal participante, ajustes de alcance durante el desarrollo y dependencia de aprobaciones administrativas para cambios relevantes.

También se reconoce que factores externos como fallas de conectividad, interrupciones eléctricas, indisponibilidad temporal de servidores o cambios en prioridades gubernamentales podrían impactar la calendarización originalmente prevista.

1.4. Roles y responsabilidades

La ejecución exitosa del Plan de Pruebas requiere la coordinación de diversos perfiles técnicos, funcionales y administrativos, cada uno con responsabilidades específicas orientadas a garantizar la calidad del sistema antes de su liberación.

El equipo de pruebas será responsable de planificar los ciclos de validación, diseñar escenarios y casos de prueba, documentar resultados, registrar incidencias y emitir recomendaciones de liberación. Por su parte, el equipo de desarrollo atenderá la corrección de defectos detectados,

apoyará el análisis técnico de incidencias y realizará pruebas unitarias previas a la entrega de componentes.

Los responsables funcionales y usuarios clave de la Dirección de Juventud participarán en la validación de reglas de negocio, revisión de flujos operativos y ejecución de pruebas de aceptación de usuario. Finalmente, el personal de tecnologías de la información apoyará con la habilitación de ambientes, accesos, respaldos y despliegues controlados.

Rol	Cantidad de recursos necesitados	Organización	Responsabilidad
Líder de pruebas	1	Proyecto Tarjeta Más Juventud	Coordinar estrategia de pruebas, seguimiento de avance, métricas y cierre de ciclos
Analista funcional	1	Proyecto Tarjeta Más Juventud	Validación de requisitos y trazabilidad funcional
Responsable del proyecto	1	H. Ayuntamiento de Xalapa	Aprobación de resultados y decisiones de liberación

TABLA 1. ROLES DEL PLAN DE PRUEBAS Y RESPONSABILIDADES ASOCIADAS

1.5. Capacitación

Con el fin de asegurar una ejecución eficiente del proceso de pruebas, será necesario proporcionar capacitación inicial a los integrantes involucrados, particularmente en el uso de herramientas, procedimientos de validación y entendimiento funcional del sistema.

El personal técnico requerirá reforzamiento en prácticas de aseguramiento de calidad, administración de incidencias, validación de *APIs* y pruebas sobre aplicaciones web responsivas. Por otro lado, los usuarios clave necesitarán capacitación orientada al uso funcional del sistema, criterios de aceptación y reporte adecuado de observaciones durante la fase de validación funcional del sistema.

Asimismo, se recomienda realizar sesiones breves previas a cada ciclo importante de pruebas, con el objetivo de alinear expectativas, revisar alcance y confirmar responsabilidades.

Tipo de entrenamiento	Número de recursos estimados	Fecha de implementación
Inducción general al Plan de Pruebas	6	Inicio de ciclo de pruebas
Uso de herramienta de incidencias y seguimiento	4	Semana 1
Validación funcional del sistema	2	Antes de prueba de aceptación
Pruebas de API y servicios web	2	Semana 2
Cierre de métricas y liberación de versión	3	Última semana

TABLA 2. NECESIDADES DE CAPACITACIÓN

2. Ambiente de pruebas y herramientas requeridas

La correcta ejecución del presente Plan de Pruebas requiere la disponibilidad de ambientes tecnológicos controlados que permitan validar el comportamiento funcional, lógico y visual del sistema “Tarjeta Más Juventud” en condiciones similares a las contempladas en su operación real. Debido a que el proyecto corresponde a una aplicación web con arquitectura cliente-servidor, las actividades de prueba deben considerar la interacción entre el *frontend*, los servicios del *backend* y la base de datos, así como los mecanismos de comunicación establecidos entre dichos componentes.

Los ambientes definidos tienen como propósito garantizar la estabilidad y confiabilidad de las pruebas unitarias, pruebas del sistema y pruebas de usabilidad contempladas dentro del alcance del proyecto. Cada ambiente posee configuraciones orientadas a validar distintos aspectos del sistema, permitiendo aislar errores, verificar integraciones y evaluar la experiencia del uso de la plataforma.

Asimismo, se contempla el uso de herramientas de soporte orientadas a la validación funcional, monitoreo de solicitudes HTTP, administración de la base de datos, y control de versiones, con el objetivo de facilitar las actividades de prueba y seguimiento de los resultados.

2.1. Ambientes de pruebas

Los ambientes de prueba definidos para el proyecto permitirán ejecutar distintos tipos de validaciones en condiciones controladas, asegurando que cada componente del sistema funcione correctamente antes de su liberación.

El ambiente de desarrollo será utilizado principalmente para pruebas unitarias y validaciones técnicas realizadas por el analista de pruebas. En este entorno se ejecutarán pruebas sobre componentes individuales, validaciones de formularios, reglas de negocio, y servicios del API.

Por otro lado, el ambiente de pruebas funcionales permitirá validar el comportamiento integral del sistema, incluyendo la comunicación entre el *frontend*, *backend* y la base de datos. En este entorno se ejecutarán pruebas de sistema orientadas a verificar flujos completos de operación, autenticación de usuarios, gestión de promociones, administración de empresas y navegación general del sistema.

Finalmente, el ambiente destinado a pruebas de usabilidad permitirá evaluar la experiencia del usuario, facilidad de navegación, claridad visual y comprensión de los procesos principales por parte de usuarios representativos del sistema. Estas pruebas serán ejecutadas utilizando navegadores web actuales y dispositivos con distintas resoluciones para garantizar una experiencia consistente.

Nombre del ambiente	Descripción	Configuración	Tipo de prueba
Ambiente de desarrollo	Entorno local utilizado para la implementación y validación inicial de los componentes del sistema.	Windows 11 Pro, Visual Studio Code, Microsoft Edge Browser.	Pruebas unitarias

Nombre del ambiente	Descripción	Configuración	Tipo de prueba
Ambiente de pruebas funcionales	Entorno destinado a validar la interacción completa entre el <i>frontend</i> , <i>backend</i> y la base de datos.	Windows 11 Pro, Visual Studio Code, Microsoft Edge Browser, conexión HTTP local.	Pruebas de sistema
Ambiente de pruebas de usabilidad	Entorno utilizado para evaluar la experiencia del usuario en términos de uso de las funciones del sistema y facilidad de navegación.	Windows 11 Pro, Visual Studio Code, Microsoft Edge Browser, Brave Browser, dispositivos con distintas resoluciones.	Pruebas de usabilidad

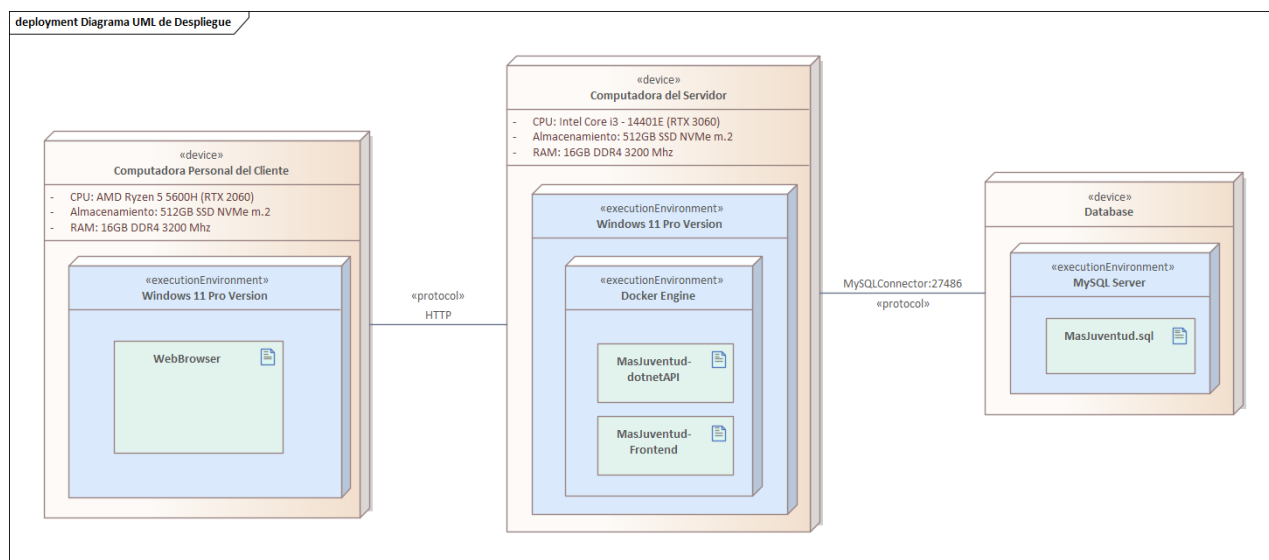
TABLA 3. AMBIENTES DE PRUEBA

2.2. Diagrama del Ambiente de desarrollo

El ambiente de desarrollo y pruebas del sistema “Tarjeta Más Juventud” se encuentra estructurado bajo una arquitectura cliente-servidor distribuida, la cual permite separar las responsabilidades correspondientes a la interfaz de usuario, lógica de negocio y almacenamiento de datos. Esta distribución facilita la mantenibilidad del sistema, mejora la escalabilidad de la solución y permite ejecutar pruebas específicas sobre cada capa tecnológica de manera independiente.

El entorno contempla un nodo cliente encargado de ejecutar la aplicación web mediante navegador, un servidor de aplicaciones donde se aloja la API desarrollada en tecnología .NET y un servidor de base de datos que administra la persistencia de la información utilizando MySQL Server. La comunicación entre cliente y servidor se realiza mediante protocolo HTTP, mientras que la interacción entre la API y la base de datos se efectúa mediante conectores especializados para MySQL.

El diagrama de despliegue asociado al presente documento representa gráficamente la distribución de los componentes, tecnologías y conexiones utilizadas durante la ejecución de las pruebas contempladas dentro del proyecto.



2.3. Herramientas de soporte

Para la ejecución adecuada del proceso de pruebas será necesario utilizar diversas herramientas tecnológicas orientadas al desarrollo, validación, monitoreo y administración de los componentes del sistema. Estas herramientas permitirán facilitar la detección de errores, análisis de incidencias, validación de servicios y documentación de resultados obtenidos durante cada ciclo de pruebas.

Las herramientas seleccionadas responden a las necesidades técnicas específicas del proyecto, considerando la arquitectura tecnológica implementada, el tipo de pruebas contempladas y la compatibilidad con las tecnologías utilizadas en el desarrollo del sistema. Asimismo, dichas herramientas permitirán mantener un entorno controlado y reproducible para la ejecución de validaciones funcionales y técnicas.

Tipo de herramienta	Nombre de la herramienta	Versión	URL
Entorno de desarrollo	Visual Studio Code	1.119	https://code.visualstudio.com/download
Herramientas de pruebas API	Postman	12	https://www.postman.com/downloads/
Herramienta de pruebas (frontend)	Vitest	4.1.6	<code>npm install -D vitest</code>
Herramienta de pruebas (frontend)	React Testing Library	Última	<code>npm install --save-dev @testing-library/react @testing-library/dom</code>
Control de versiones	Git	2.54.0	https://git-scm.com/install/
Navegador web	Microsoft Edge	Última	https://www.microsoft.com/edge

TABLA 4. HERRAMIENTAS DE SOPORTE

3. Estrategia general de las pruebas

La estrategia general de pruebas definida para el sistema “Tarjeta Más Juventud” tiene como propósito establecer los lineamientos, actividades y criterios que permitirán validar el correcto funcionamiento de la solución tecnológica antes de su liberación. Esta estrategia contempla la planificación estructurada de las actividades de aseguramiento de calidad, considerando los distintos tipos de pruebas definidos dentro del alcance del proyecto, así como los recursos técnicos y humanos necesarios para su ejecución.

Debido a que el sistema corresponde a una plataforma web con arquitectura cliente-servidor, las actividades de prueba se enfocarán en validar tanto componentes individuales como el comportamiento integral de la aplicación. Para ello, se ejecutarán pruebas unitarias orientadas a verificar funcionalidades específicas del sistema, pruebas de sistema destinadas a evaluar el comportamiento completo de los flujos funcionales y pruebas de usabilidad enfocadas en la experiencia de navegación e interacción del usuario final.

La estrategia adoptada busca identificar defectos en etapas tempranas del desarrollo, reducir riesgos asociados a fallos funcionales y garantizar que los requisitos definidos para el sistema sean implementados correctamente. Asimismo, se pretende asegurar que la plataforma proporcione una experiencia de uso clara, estable y consistente para los usuarios administrativos y ciudadanos beneficiarios contemplados dentro del proyecto.

3.1. Fases de prueba

El proceso de pruebas del sistema estará conformado por distintas fases orientadas a garantizar una ejecución estructurada y controlada de las actividades de validación. Cada fase contempla objetivos específicos, actividades definidas y entregables asociados que permitirán mantener trazabilidad sobre el avance del aseguramiento de calidad del proyecto.

Las fases consideradas dentro del presente plan incluyen la planeación de pruebas, definición de pruebas y ejecución de pruebas. Estas etapas permitirán organizar adecuadamente las actividades relacionadas con diseño de casos de prueba, validación funcional, documentación de incidencias y elaboración de resultados finales.

3.1.1. Planeación de prueba

La fase de planeación de pruebas tiene como finalidad definir el alcance general del proceso de validación, estableciendo los tipos de pruebas que serán ejecutados, los componentes involucrados, los recursos necesarios, los ambientes tecnológicos requeridos y la calendarización de las actividades correspondientes.

Durante esta etapa se identifican las funcionalidades críticas del sistema, los módulos que serán sometidos a validación y los criterios generales de aceptación aplicables a cada tipo de prueba. Asimismo, se establecen los roles y responsabilidades de los participantes involucrados en el proceso, incluyendo desarrolladores, responsables funcionales y personal encargado del seguimiento de incidencias.

La planeación también contempla la definición de ambientes de prueba, herramientas de soporte, mecanismos de documentación y estrategias para el control de defectos detectados durante la ejecución. De igual forma, se consideran factores de riesgo que puedan afectar el desarrollo de las pruebas, tales como limitaciones de tiempo, disponibilidad de infraestructura o cambios de alcance durante el proyecto.

El principal entregable de esta fase corresponde al presente documento denominado “Plan de Pruebas”, el cual concentra la estrategia general, lineamientos y organización de las actividades de aseguramiento de calidad del sistema.

Entregable: Documento de Plan de Pruebas

3.1.2. Definición de pruebas

La fase de definición de pruebas consiste en la elaboración detallada de los escenarios y casos de prueba que serán utilizados durante las actividades de validación funcional y técnica del sistema. En esta etapa se documentan los procedimientos necesarios para verificar el cumplimiento de los requisitos funcionales y flujos operativos definidos para la plataforma.

Los casos de prueba serán redactados utilizando una estructura basada en acciones y resultados esperados, permitiendo establecer de forma clara las condiciones necesarias para validar cada funcionalidad del sistema. Cada caso incluirá información relacionada con identificador, objetivo, precondiciones, pasos de ejecución, datos de entrada y comportamiento esperado.

Asimismo, durante esta etapa se definirán los conjuntos de datos de prueba requeridos para validar registros, autenticación, operaciones CRUD, generación de promociones, consultas de información y navegación dentro del sistema. Esto permitirá asegurar consistencia durante la ejecución de pruebas y facilitar la reproducción de escenarios específicos en caso de detectar incidencias.

La definición de pruebas también contempla la asociación entre requisitos funcionales, módulos del sistema y casos de prueba correspondientes, permitiendo mantener trazabilidad sobre la cobertura de validación aplicada al proyecto.

Entregables: Documento consolidado de casos de prueba, conjunto de datos de prueba, matriz de trazabilidad de pruebas.

3.1.3. Ejecución de pruebas

La fase de ejecución de pruebas corresponde al proceso mediante el cual se ponen en práctica los casos de prueba previamente definidos, con el objetivo de validar el comportamiento real del sistema y comparar los resultados obtenidos contra los resultados esperados documentados.

Durante esta etapa se ejecutarán pruebas unitarias, pruebas de sistema y pruebas de usabilidad sobre los distintos módulos y componentes del sistema “Tarjeta Más Juventud”. Cada ejecución permitirá identificar errores funcionales, inconsistencias visuales, fallos de integración o comportamientos no previstos dentro de la plataforma.

Cuando se detecte una discrepancia entre el comportamiento esperado y el comportamiento observado, se registrará una incidencia especificando información relacionada con severidad, prioridad, descripción del problema, evidencia asociada y componente afectado. Posteriormente, dichas incidencias serán atendidas por el equipo de desarrollo para su corrección y posterior validación.

Asimismo, durante esta fase se documentarán resultados, métricas y observaciones relevantes derivadas de cada ciclo de pruebas, permitiendo evaluar el nivel de calidad alcanzado por el sistema y determinar si las funcionalidades cumplen con los criterios establecidos previamente.

Al finalizar la ejecución de pruebas se elaborará un reporte de resultados que concentrará información relacionada con casos ejecutados, incidencias detectadas, porcentaje de aprobación y estado general de validación del sistema.

Entregables: Reporte de ejecución de pruebas, registro de incidencias, evidencia de pruebas, reporte final de resultados.

3.2. Estrategia de pruebas

La estrategia de pruebas del proyecto se encuentra alineada con el ciclo de desarrollo definido para el sistema “Tarjeta Más Juventud”, permitiendo ejecutar actividades de validación de manera progresiva conforme los módulos y funcionalidades son implementados. Esta estrategia busca garantizar que cada componente del sistema sea evaluado individual y posteriormente integrado dentro de flujos completos de operación.

Las actividades de prueba serán organizadas en ciclos iterativos asociados al avance del desarrollo del sistema. Inicialmente se ejecutarán pruebas unitarias sobre componentes específicos tales como formularios, servicios API, validaciones, funciones auxiliares y reglas de negocio. Posteriormente se realizarán pruebas de sistema orientadas a validar la interacción integral entre *frontend*, *backend* y base de datos, incluyendo autenticación, administración de empresas, registro de sucursales, gestión de promociones y navegación general.

Finalmente, se ejecutarán pruebas de usabilidad enfocadas en evaluar la claridad visual de la interfaz, facilidad de navegación, comprensión de procesos y experiencia general de interacción de los usuarios finales con la plataforma. Estas pruebas permitirán identificar oportunidades de mejora relacionadas con diseño, accesibilidad funcional y organización de elementos visuales.

3.2.1. Criterios de entrada, salida y suspensión de pruebas

Para iniciar formalmente la ejecución de pruebas será necesario cumplir una serie de criterios mínimos relacionados con disponibilidad de funcionalidades, ambientes y documentación requerida. Entre los principales criterios de entrada se encuentran la finalización parcial o total del desarrollo de los módulos a validar, disponibilidad de ambientes de prueba funcionales, acceso a herramientas necesarias y definición previa de casos de prueba y datos asociados.

Los criterios de salida estarán determinados por el cumplimiento satisfactorio de los casos de prueba definidos, la corrección de incidencias críticas detectadas y la validación general de los

flujos funcionales establecidos dentro del alcance del proyecto. Asimismo, se considerará como criterio de salida la generación de evidencia documental y reportes finales de resultados.

Las actividades de prueba podrán suspenderse temporalmente cuando existan condiciones que impidan continuar con la validación de manera confiable, tales como fallas críticas en infraestructura, indisponibilidad de ambientes, pérdida de conectividad, defectos bloqueantes que afecten múltiples funcionalidades o ausencia de información necesaria para ejecutar los casos de prueba correspondientes.

En caso de suspensión, las actividades serán reanudadas una vez que las condiciones identificadas hayan sido corregidas y exista autorización para continuar con el ciclo de validación.

3.2.2. Trazabilidad de las pruebas

La trazabilidad de las pruebas permitirá mantener una relación clara entre requisitos funcionales, módulos del sistema, casos de uso y casos de prueba ejecutados durante el proceso de validación. Este mecanismo facilitará identificar qué funcionalidades han sido probadas, cuáles presentan incidencias y qué componentes aún requieren validación.

Para garantizar dicha trazabilidad se utilizará una matriz de seguimiento que relacionará cada requisito funcional con sus respectivos escenarios y casos de prueba asociados. Esta estructura permitirá verificar la cobertura del proceso de pruebas y asegurar que las funcionalidades prioritarias del sistema hayan sido evaluadas correctamente.

Asimismo, la trazabilidad facilitará el seguimiento de incidencias detectadas, permitiendo asociar defectos específicos con los módulos afectados y las pruebas ejecutadas durante cada ciclo de validación.

3.2.3. Problemas de las pruebas

Durante la ejecución de pruebas podrán presentarse incidencias técnicas, funcionales u operativas que afecten parcial o totalmente el comportamiento esperado del sistema. Dichas incidencias incluirán errores de validación, fallos de integración, comportamientos inconsistentes, problemas visuales, errores de conexión o defectos relacionados con reglas de negocio.

Cada problema identificado será registrado mediante un mecanismo de seguimiento de incidencias que incluirá información relacionada con identificador, descripción del defecto, severidad, prioridad, módulo afectado, pasos para reproducir el error y evidencia asociada. Posteriormente, el equipo de desarrollo analizará la incidencia y realizará las correcciones necesarias antes de una nueva validación.

Las incidencias serán clasificadas según su nivel de impacto dentro del sistema, priorizando la atención de errores críticos que impidan la operación normal de funcionalidades esenciales. Una vez aplicada la corrección correspondiente, se ejecutarán nuevamente los casos de prueba relacionados con el objetivo de verificar la solución implementada.

3.2.4. Principios de las pruebas

El proceso de pruebas del sistema se desarrollará bajo una serie de principios orientados a fortalecer la calidad de las validaciones realizadas y optimizar la detección temprana de defectos. Estos principios servirán como guía general para la planificación y ejecución de las actividades contempladas dentro del presente plan.

En primer lugar, se priorizarán las funcionalidades críticas del sistema, especialmente aquellas relacionadas con autenticación, gestión de promociones, administración de empresas y persistencia de información. Asimismo, las pruebas serán planificadas desde etapas tempranas del desarrollo, permitiendo identificar errores antes de la integración completa de los módulos.

Otro principio fundamental será la ejecución progresiva de validaciones, iniciando con componentes individuales y avanzando posteriormente hacia flujos integrales de operación. De igual forma, se buscará mantener ambientes de prueba controlados y datos consistentes para garantizar resultados reproducibles y confiables.

Finalmente, se promoverá la documentación continua de incidencias, resultados y evidencias de prueba, permitiendo mantener trazabilidad sobre el proceso de aseguramiento de calidad y facilitando futuras actividades de mantenimiento o evolución del sistema.

4. Pruebas unitarias

Las pruebas unitarias del proyecto “Tarjeta Más Juventud” se realizaron sobre el *frontend* del sistema, con el propósito de validar de manera aislada el comportamiento de componentes lógicos, *hooks* personalizados, utilidades de autenticación, manejo de almacenamiento local, administración de caché y estructuras básicas de páginas. Estas pruebas formaron parte del proceso de aseguramiento de calidad del sistema y permitieron comprobar que determinadas unidades de código respondieran correctamente antes de ser evaluadas dentro de flujos funcionales más amplios durante las pruebas de sistema y usabilidad.

Para la ejecución de estas pruebas se utilizó **Vitest 4.1.6** como *framework* principal, complementado con **React Testing Library** para la validación de *hooks* y componentes asociados al entorno **React**. La elección de estas herramientas permitió ejecutar pruebas automatizadas dentro de un entorno controlado, reproducible y compatible con la estructura tecnológica del *frontend*. El registro técnico generado durante el proceso documentó un total de veinte escenarios principales de prueba, relacionados tanto con validaciones funcionales como con ajustes de configuración, corrección de errores y estabilización del entorno de ejecución.

El proceso no se limitó únicamente a ejecutar pruebas ya existentes, sino que implicó una fase previa de diagnóstico y corrección del entorno. Inicialmente, el proyecto contaba con una configuración de pruebas amplia, con más de cuatrocientas pruebas potenciales, pero dicha configuración provocó problemas de agotamiento de memoria durante la ejecución. Como resultado, fue necesario analizar la causa raíz, revisar archivos de configuración, ajustar el entorno de ejecución y reducir la suite activa hacia un conjunto más estable de pruebas esenciales.

Uno de los hallazgos más relevantes fue la presencia de excepciones no controladas en el *hook* `useFetchData.js`, particularmente por el uso de una instrucción `throw err` después del manejo del error en el bloque `catch`. Esta situación provocaba rechazos de promesas no controlados que se acumulaban durante la ejecución de pruebas, incrementando el consumo de memoria y generando fallos en cascada. La corrección consistió en manejar el error exclusivamente mediante el estado interno del *hook*, evitando relanzar la excepción después de haber sido procesada.

También se realizaron ajustes sobre la configuración general de **Vitest**, especialmente para adaptarla a la versión 4 del *framework*. Entre las modificaciones aplicadas se incluyó el cambio de la configuración anterior basada en `poolOptions.threads.singleThread` hacia la opción `singleThread: true`, así como la modificación del tipo de pool hacia `forks`, lo cual favoreció la estabilidad en entornos Windows. Además, se estableció el entorno `node` como predeterminado y se habilitó `jsdom` únicamente en los archivos que realmente requerían funcionalidades del navegador, como `localStorage`, renderizado de *hooks* o interacción con objetos del DOM.

Durante el proceso también se incorporaron mecanismos específicos para evitar errores derivados del acceso a objetos propios del navegador en entornos donde no estaban disponibles. En particular, se modificó el archivo `vitest.setup.js` para validar la existencia de `window` antes de aplicar configuraciones como `matchMedia`. Esto permitió que los archivos ejecutados en entorno `node` no fallaran por referencias inexistentes, al mismo tiempo que los archivos ejecutados en `jsdom` conservaran los *mocks* necesarios para simular el comportamiento del navegador.

Otra parte importante del proceso fue la corrección de pruebas con comportamiento asíncrono. Algunos *hooks* presentaban fallos en las aserciones debido a que las llamadas a `act()` y `waitFor()` no esperaban correctamente la resolución de promesas. Para resolverlo, se ajustaron las pruebas relacionadas con *hooks*, especialmente aquellas vinculadas con registro, OCR y obtención de datos, garantizando que las operaciones asíncronas fueran esperadas adecuadamente antes de validar resultados.

Como resultado de estas acciones, la suite de pruebas quedó organizada en una versión más ligera y estable, con aproximadamente entre 80 y 100 pruebas activas enfocadas en funcionalidades esenciales. Aunque algunos archivos de prueba fueron excluidos temporalmente por su alto consumo de memoria, se conservó la cobertura de componentes prioritarios relacionados con autenticación, tokens, *hooks* de datos, paginación, *dashboard* y caché.

4.1. Objetivos

El objetivo general de las pruebas unitarias fue verificar que las unidades individuales del *frontend* funcionaran correctamente de manera aislada, antes de evaluar su comportamiento dentro de flujos completos del sistema. Esto permitió reducir el riesgo de que errores internos en *hooks*, utilidades o configuraciones afectaran posteriormente las pruebas de sistema o la experiencia del usuario final.

De manera específica, las pruebas buscaron comprobar el comportamiento de las funciones de autenticación, validando que los tokens, los datos de usuario y las operaciones de cierre de sesión se administraran correctamente mediante *localStorage*. Estas validaciones fueron relevantes porque la autenticación constituye una de las bases funcionales del sistema, al controlar el acceso a módulos administrativos y proteger la información asociada a usuarios registrados.

Otro objetivo fue validar los *hooks* personalizados utilizados para obtener, procesar y administrar información dentro de la aplicación. En particular, se probaron *hooks* relacionados con carga de datos, paginación de listas, combinación de datos para *dashboards* y manejo de estados de error o carga. Estas unidades son importantes porque concentran lógica reutilizable que puede impactar varias vistas del sistema.

También se buscó comprobar que el sistema manejara adecuadamente escenarios de error, especialmente en operaciones asíncronas. Esto incluyó la validación del comportamiento esperado cuando una solicitud falla, cuando un dato no existe, cuando el contenido almacenado no tiene formato válido o cuando una operación depende de un entorno de navegador simulado.

Un objetivo adicional consistió en estabilizar la infraestructura de pruebas automatizadas. Durante el proceso se identificaron problemas de memoria, configuración y compatibilidad con *Vitest 4*, por lo que parte de la validación incluyó asegurar que el entorno pudiera ejecutar las pruebas sin fallos por configuración, advertencias obsoletas o referencias incorrectas a objetos del navegador.

Finalmente, las pruebas unitarias tuvieron como propósito dejar una base técnica reutilizable para futuras iteraciones del proyecto. La configuración final permitió ejecutar pruebas esenciales de forma más rápida y estable, facilitando que nuevos cambios en el *frontend* puedan validarse sin depender exclusivamente de pruebas manuales.

4.1.1. Alcance

El alcance de las pruebas unitarias se concentró en el *frontend* del sistema “Tarjeta Más Juventud”, específicamente en unidades de código implementadas en *React* y *JavaScript*. No se consideró dentro de esta fase la validación completa de flujos de negocio de extremo a extremo, ya que dichos escenarios corresponden a pruebas de sistema. Tampoco se incluyó la validación formal de rendimiento, seguridad o accesibilidad, debido a que estas pruebas no forman parte del alcance definido para esta etapa del plan.

Las unidades principales incluidas en el alcance fueron los módulos de autenticación, almacenamiento de sesión, *hooks* personalizados, funciones de caché y archivos básicos de prueba asociados a páginas principales. En el caso de autenticación, se consideraron funciones como `getToken()`, `getUser()`, `isAuthenticated()` y `logout()`, con el objetivo de comprobar el manejo correcto de tokens, usuarios y cierre de sesión dentro del almacenamiento local.

También se incluyeron pruebas sobre funciones relacionadas con recuperación y limpieza de datos de sesión, principalmente mediante el archivo `getToken.test.js`. Estas pruebas validaron operaciones sobre *localStorage*, incluyendo recuperación de token, recuperación de usuario, eliminación de información y comportamiento ante datos inexistentes o inválidos.

Dentro del alcance también se consideró el *hook* `useFetchData`, encargado de realizar solicitudes de datos, administrar estados de carga, manejar errores y utilizar mecanismos de caché. Este componente fue especialmente relevante porque durante el proceso se detectó un problema de excepciones no controladas que afectaba la estabilidad general de las pruebas. La corrección de este *hook* formó parte esencial del alcance técnico de las pruebas unitarias.

Asimismo, se incluyó el *hook* `useListPagination`, utilizado para gestionar controles de paginación, navegación entre elementos y cálculo de páginas. Este *hook* fue validado mediante múltiples pruebas orientadas a asegurar que los estados internos se actualizarán correctamente durante la navegación.

El *hook* `useDashboardData` también fue considerado dentro del alcance, debido a su función de combinar información proveniente de múltiples fuentes de datos. Las pruebas asociadas verificaron que la información del `dashboard` pudiera agregarse correctamente, manejar estados de carga y responder ante errores de distintas fuentes.

Se incluyó además la utilidad `apiCache`, cuyo propósito es almacenar, recuperar y eliminar datos en memoria durante la ejecución de la aplicación. La validación de esta unidad fue importante para confirmar que la caché operara de forma consistente con distintos tipos de datos y no afectara la estabilidad de las pruebas.

Finalmente, se crearon o corrigieron pruebas mínimas para las páginas `Login` y `HomePage`. Estas pruebas tuvieron un alcance básico, orientado a garantizar que los archivos existieran y contaran con una estructura válida para *Vitest*, evitando errores por archivos vacíos o inexistentes durante la ejecución de la suite.

4.1.2. Casos de prueba

Los casos de prueba unitarios documentados reflejan tanto validaciones funcionales como ajustes técnicos necesarios para estabilizar la ejecución automatizada. Esta decisión se tomó debido a que, durante el proceso, varios errores no estaban relacionados directamente con la lógica de negocio, sino con la configuración del *framework*, el entorno de ejecución, el consumo de memoria y la compatibilidad entre *Node.js* y *jsdom*.

La siguiente tabla resume los principales escenarios ejecutados y documentados como parte del histórico de pruebas unitarias del proyecto.

ID	Precondicion(es)	Pasos	Resultado esperado	Resultado obtenido	Estado
TEST-01	Vitest 4.1.6 instalado y React Testing Library configurado	Configurar la ejecución inicial de Vitest con ambiente jsdom para más de 400 pruebas	Todas las pruebas deben cargar y ejecutarse correctamente	La configuración fue creada, pero se presentó agotamiento de memoria aproximadamente al llegar a 2 GB de heap	Fallido por memoria
TEST-02	Se identificó agotamiento de memoria durante la ejecución secuencial	Analizar la causa raíz del fallo de memoria y revisar errores acumulados durante la ejecución	Identificar el origen principal de los fallos en cascada	Se localizó que useFetchData.js relanzaba errores después del bloque catch, generando promesas rechazadas no controladas	Resuelto
TEST-03	Se detectó el problema en el manejo de errores de useFetchData.js	Eliminar la instrucción throw err y conservar el manejo mediante estado setError()	El error debe gestionarse mediante estado sin relanzar la excepción	El manejo de errores fue corregido y dejaron de generarse rechazos de promesas no controlados	Aprobado
TEST-04	Vitest mostraba advertencias por configuración obsoleta	Actualizar la configuración de poolOptions.threads.single Thread hacia singleThread: true	Eliminar advertencias de obsolescencia en la salida de consola	vitest.config.js fue actualizado con sintaxis compatible con Vitest 4	Aprobado
TEST-05	Existían errores por referencias a window en entorno Node.js	Agregar validación typeof window !== "undefined" en vitest.setup.js	El archivo de configuración debe funcionar tanto en Node.js como en jsdom	El mock de window.matchMedia se aplica solo cuando existe window; Node.js lo ignora sin fallar	Aprobado
TEST-06	Algunas pruebas de hooks fallaban por operaciones asíncronas no esperadas	Corregir llamadas a act() y waitFor() usando await cuando fuera necesario	Las aserciones deben ejecutarse después de resolver las promesas	Las pruebas de hooks fueron ajustadas para evitar condiciones de carrera	Aprobado
TEST-07	NODE_OPTIONS no se aplicaba correctamente en scripts npm sobre Windows	Crear archivo test-runner.bat para configurar variables de memoria antes de ejecutar Vitest	El entorno debe iniciar con memoria suficiente antes de correr las pruebas	El archivo batch configuró correctamente opciones de memoria para la ejecución en Windows	Aprobado
TEST-08	Las pruebas de autenticación fallaban porque localStorage no existe en entorno Node.js	Agregar directiva @vitest-environment jsdom a archivos de autenticación y tokens	localStorage debe estar disponible durante la ejecución de pruebas	Los archivos auth.test.js y getToken.test.js se ejecutan correctamente en jsdom	Aprobado

ID	Precondicion(es)	Pasos	Resultado esperado	Resultado obtenido	Estado
TEST-09	La ejecución de más de 400 pruebas seguía generando agotamiento de memoria	Implementar estrategia mixta: entorno Node.js por defecto y jsdom solo en archivos necesarios	Reducir el consumo de memoria evitando jsdom global	Se redujo significativamente el consumo de memoria, aunque la acumulación secuencial aún podía provocar fallos	Parcial
TEST-10	La ejecución secuencial acumulaba demasiado estado en un solo proceso	Reducir la suite activa de más de 400 pruebas a un conjunto esencial de 80 a 100 pruebas	Ejecutar únicamente pruebas críticas sin provocar fallos de memoria	Se implementó una lista de exclusión en vitest.config.js para archivos pesados	Aprobado
TEST-11	Se identificaron archivos con alto número de pruebas y consumo elevado	Excluir temporalmente archivos como useRegistro, useRegister, useOcr, formHelpers, validations y extras	Reducir carga de ejecución y conservar estabilidad	Los archivos pesados fueron excluidos correctamente mediante configuración	Aprobado
TEST-12	HomePage.test.js estaba vacío y generaba error “No test suite found”	Agregar una prueba mínima con describe e it	El archivo debe contener una estructura válida de prueba	HomePage.test.js fue corregido con una prueba mínima de existencia	Aprobado
TEST-13	No existía el archivo Login.test.js esperado por la configuración	Crear archivo Login.test.js con estructura mínima de prueba	El archivo debe existir y contener una suite válida	El archivo fue creado correctamente con una prueba placeholder	Aprobado
TEST-14	La configuración reducida quedó preparada para ejecución	Ejecutar npm run test o test-runner.bat con la suite esencial	Todas las pruebas activas deben ejecutarse sin fallos de memoria	La configuración quedó lista y pendiente de verificación final de ejecución completa	Aprobado
TEST-15	Existe el módulo de utilidades de autenticación	Ejecutar pruebas sobre getToken(), getUser(), isAuthenticated() y logout()	Las funciones deben recuperar, validar y eliminar datos de autenticación correctamente	Se documentaron 22 pruebas completas para operaciones de autenticación y sesión	Aprobado
TEST-16	Existe el módulo de almacenamiento de token y usuario	Validar funciones de recuperación y limpieza de datos de sesión en localStorage	Las funciones deben operar correctamente con persistencia local	Se documentaron 21 pruebas sobre manejo de token, usuario y limpieza de autenticación	Aprobado
TEST-17	Existe el hook useFetchData y el módulo de caché API	Validar carga de datos, caché, manejo de errores y reintentos	El hook debe obtener datos, cachear respuestas y manejar errores sin fallar	Se documentaron 11 pruebas sobre obtención de datos y manejo de estados	Aprobado
TEST-18	Existe el hook useListPagination	Validar controles de scroll, navegación de listas y cálculo de páginas	El hook debe actualizar estados de paginación correctamente	Se documentaron 19 pruebas sobre navegación y control de paginación	Aprobado
TEST-19	Existe el hook useDashboardData	Validar combinación de múltiples fuentes de datos para dashboard	El hook debe cargar y combinar información correctamente	Se documentaron 8 pruebas sobre agregación de datos de dashboard	Aprobado
TEST-20	Existe la utilidad apiCache	Validar operaciones get(), set() y delete() con diferentes tipos de datos	La caché debe almacenar, recuperar y eliminar datos correctamente	Las pruebas de apiCache quedaron consideradas dentro de la suite activa	Aprobado

TABLA 5. CASOS DE PRUEBA PARA LAS PRUEBAS UNITARIAS

Para la documentación de las pruebas unitarias se consideró una organización por módulo funcional, con el objetivo de presentar de manera clara qué componentes fueron evaluados, qué comportamiento se esperaba validar y cuál fue el resultado obtenido. La suite activa quedó conformada por aproximadamente 107 pruebas automatizadas ejecutadas mediante *Vitest 4.1.6* y *React Testing Library*, distribuidas en módulos de autenticación, almacenamiento, obtención de datos, caché, paginación, *dashboard* y componentes de página.

Pruebas de autenticación — `auth.test.js`

El archivo `auth.test.js` valida las funciones asociadas al proceso de autenticación y administración de sesión del *frontend*. Las pruebas se enfocaron en comprobar la recuperación del token, recuperación del usuario autenticado, validación del estado de autenticación, detección de tokens inválidos o expirados y cierre seguro de sesión. Este módulo es relevante debido a que controla el acceso lógico del usuario a las funcionalidades protegidas del sistema.

ID	Funcionalidad	Escenario validado	Resultado esperado	Estado
AUTH-001	<code>getToken()</code>	Recuperar token existente	Retornar token almacenado	Aprobado
AUTH-002	<code>getToken()</code>	<code>localStorage</code> vacío	Retornar null	Aprobado
AUTH-003	<code>getToken()</code>	Token no definido	Retornar null	Aprobado
AUTH-004	<code>getUser()</code>	Usuario parseado correctamente	Retornar objeto usuario	Aprobado
AUTH-005	<code>getUser()</code>	Usuario inexistente	Retornar null	Aprobado
AUTH-006	<code>getUser()</code>	JSON inválido	Retornar null	Aprobado
AUTH-007	<code>isAuthenticated()</code>	Token válido y no expirado	Retornar true	Aprobado
AUTH-008	<code>isAuthenticated()</code>	Token expirado	Retornar false	Aprobado
AUTH-009	<code>isAuthenticated()</code>	Token inválido o ausente	Retornar false	Aprobado
AUTH-010	<code>logout()</code>	Cierre completo de sesión	Eliminar token y usuario	Aprobado

TABLA 6. PRUEBAS UNITARIAS DE AUTENTICACIÓN

Pruebas de almacenamiento — `getToken.test.js`

El archivo `getToken.test.js` valida las operaciones relacionadas con almacenamiento, recuperación y limpieza de credenciales dentro de `localStorage`. Estas pruebas permitieron verificar que la información de sesión se mantenga correctamente durante la interacción del usuario y que pueda eliminarse de manera segura cuando se ejecuta el cierre de sesión.

ID	Funcionalidad	Escenario validado	Resultado esperado	Estado
STORE-001	<code>getToken()</code>	Recuperar token desde <code>localStorage</code>	Token disponible	Aprobado
STORE-002	<code>getUser()</code>	Recuperar usuario parseado	Objeto usuario completo	Aprobado
STORE-003	<code>clearAuth()</code>	Limpiar datos de sesión	<code>localStorage</code> vacío	Aprobado

STORE-004	localStorage	Persistencia entre llamadas	Datos conservados correctamente	Aprobado
STORE-005	localStorage	Datos corruptos	Manejo sin interrupción del sistema	Aprobado
STORE-006	Ciclo completo	Login y logout	Sesión limpiada completamente	Aprobado
STORE-007	Cambio de usuario	Reemplazo de datos previos	Datos anteriores reemplazados	Aprobado

TABLA 7. PRUEBAS UNITARIAS DE ALMACENAMIENTO

Pruebas de obtención de datos — useFetchData.test.js

El archivo `useFetchData.test.js` valida el comportamiento del *hook* encargado de realizar solicitudes de datos, administrar estados de carga, almacenar respuestas en caché y manejar errores. Este componente fue relevante dentro del proceso porque concentra lógica reutilizable de comunicación con servicios y manejo de datos dentro del *frontend*.

ID	Funcionalidad	Escenario validado	Resultado esperado	Estado
FETCH-001	useFetchData()	Inicialización del hook	Estado inicial válido	Aprobado
FETCH-002	Fetching	Obtención exitosa de datos	Respuesta procesada correctamente	Aprobado
FETCH-003	Caché	Almacenar respuesta	Siguiente llamada utiliza caché	Aprobado
FETCH-004	Manejo de error	API retorna error	Estado de error actualizado	Aprobado
FETCH-005	Reintento	Fallo y recuperación	Reintento ejecutado correctamente	Aprobado
FETCH-006	Invalidación	Limpiar caché	Nueva solicitud obtiene datos recientes	Aprobado
FETCH-007	Estado de carga	Solicitud en proceso	isLoading = true	Aprobado
FETCH-008	Recuperación	Nuevo intento posterior al error	El hook permite nueva ejecución	Aprobado

TABLA 8. PRUEBAS UNITARIAS DE OBTENCIÓN DE DATOS

Pruebas de paginación — useListPagination.test.js

El archivo `useListPagination.test.js` valida el *hook* utilizado para controlar navegación entre páginas, límites de desplazamiento, cálculo de índices y actualización de estado. Estas pruebas permitieron confirmar que las listas paginadas del sistema se comportan correctamente ante acciones de avance, retroceso y navegación directa.

ID	Funcionalidad	Escenario validado	Resultado esperado	Estado
PAGI-001	Inicialización	Estado inicial	Página inicial e índices correctos	Aprobado
PAGI-002	Siguiente	Avanzar página	Página incrementada	Aprobado

PAGI-003	Anterior	Retroceder página	Página decrementada	Aprobado
PAGI-004	Límite superior	Avanzar desde última página	Permanecer en última página	Aprobado
PAGI-005	Límite inferior	Retroceder desde primera página	Permanecer en primera página	Aprobado
PAGI-006	Navegación directa	Ir a una página específica	Estado actualizado correctamente	Aprobado

TABLA 9. PRUEBAS UNITARIAS DE PAGINACIÓN

Pruebas de dashboard — useDashboardData.test.js

El archivo `useDashboardData.test.js` valida el *hook* encargado de agregar datos provenientes de múltiples fuentes para su presentación en el *dashboard*. Las pruebas comprobaron la carga de información, sincronización de estados, manejo de errores parciales y transformación de datos.

ID	Funcionalidad	Escenario validado	Resultado esperado	Estado
DASH-001	Agregación	Cargar múltiples fuentes	Datos disponibles correctamente	Aprobado
DASH-002	Estado de carga	Solicitudes en ejecución	<code>isLoading = true</code>	Aprobado
DASH-003	Errores parciales	Una fuente falla	Las demás fuentes siguen funcionando	Aprobado
DASH-004	Sincronización	Finalización de carga	Datos sincronizados correctamente	Aprobado

TABLA 10. PRUEBAS UNITARIAS DEL DASHBOARD

Pruebas de caché API — apiCache.test.js

El archivo `apiCache.test.js` valida las operaciones básicas de almacenamiento temporal de datos en caché. Estas pruebas permitieron comprobar que la utilidad de caché puede guardar, recuperar y eliminar entradas de forma consistente, reduciendo solicitudes innecesarias y apoyando la eficiencia de los *hooks* de datos.

ID	Funcionalidad	Escenario validado	Resultado esperado	Estado
CACHE-001	<code>get()</code>	Recuperar entrada existente	Valor correcto retornado	Aprobado
CACHE-002	<code>set()</code>	Guardar entrada	Valor almacenado correctamente	Aprobado
CACHE-003	<code>delete()</code>	Eliminar entrada	Entrada removida de la caché	Aprobado
CACHE-004	<code>get()</code>	Entrada inexistente	Retornar <code>undefined</code>	Aprobado

TABLA 11. PRUEBAS UNITARIAS DE CACHÉ API

Pruebas de Login — Login-simple.test.js

El archivo `Login-simple.test.js` valida aspectos básicos del formulario de inicio de sesión, incluyendo renderizado, validación de entradas, envío de credenciales, manejo de errores y flujo posterior a un inicio de sesión exitoso. Aunque estas pruebas se aproximan a una validación de

integración ligera, se incluyeron dentro de la suite unitaria por su bajo alcance y por validar componentes concretos del *frontend*.

ID	Funcionalidad	Escenario validado	Resultado esperado	Estado
LOGIN-001	Renderizado	Visualización del formulario	Inputs y botón visibles	Aprobado
LOGIN-002	Validación	Envío vacío	Mostrar error de validación	Aprobado
LOGIN-003	Envío	Credenciales correctas	Ejecutar llamada de autenticación	Aprobado
LOGIN-004	Error	Credenciales incorrectas	Mostrar mensaje de error	Aprobado
LOGIN-005	Flujo completo	Login exitoso	Ejecutar redirección	Aprobado

TABLA 12. PRUEBAS UNITARIAS DEL LOGIN

4.1.3. Fuentes de datos

Las fuentes de datos utilizadas en las pruebas unitarias fueron principalmente datos simulados, estructuras *mock* y almacenamiento temporal dentro del entorno de pruebas. Esta decisión permitió validar el comportamiento de unidades individuales sin depender de servicios externos, bases de datos productivas o disponibilidad de infraestructura institucional.

Para las pruebas relacionadas con autenticación se utilizó `localStorage` simulado mediante el entorno *jsdom*. Este mecanismo permitió almacenar y recuperar tokens, objetos de usuario y datos de sesión, reproduciendo el comportamiento esperado del navegador durante la operación real del sistema.

En las pruebas de *hooks*, especialmente `useFetchData`, se emplearon respuestas simuladas de servicios API mediante *mocks*. Esto permitió comprobar escenarios de carga exitosa, errores de solicitud, almacenamiento en caché y recuperación posterior de información sin realizar peticiones reales hacia un *backend*.

Para la validación de paginación se utilizaron arreglos de datos de prueba. Estos permitieron simular listas de elementos, cambios de página, navegación entre posiciones y actualización de estados internos. Esta fuente fue suficiente para comprobar que la lógica del *hook* respondiera correctamente ante diferentes tamaños de colección y movimientos del usuario.

En el caso de `useDashboardData`, se utilizaron objetos simulados que representaban múltiples fuentes de información combinadas. Esto permitió evaluar el comportamiento del *hook* al consolidar datos, manejar estados de carga y responder ante escenarios de error parcial.

Las pruebas de caché utilizaron datos en memoria, incluyendo diferentes tipos de valores, con el objetivo de validar que la utilidad `apiCache` pudiera almacenar, recuperar y eliminar información correctamente. Esta fuente permitió comprobar el comportamiento interno de la caché sin depender de archivos, red o base de datos.

Además, se utilizaron archivos de configuración como fuente operativa para la ejecución de pruebas. `vitest.config.js` definió el alcance de archivos incluidos y excluidos, el entorno predeterminado, el tipo de pool y los tiempos máximos de ejecución. Por su parte,

`vitest.setup.js` permitió preparar *mocks* globales y limpieza entre pruebas. Finalmente, `test-runner.bat` funcionó como fuente de configuración para variables de memoria en Windows.

Requisitos de dato para la prueba	Fuente	Puntos de contacto
Token de autenticación válido	localStorage simulado mediante jsdom	Archivo <code>auth.test.js</code> ; módulo <code>auth.js</code>
Token inexistente, inválido o expirado	Datos mock almacenados o ausentes en localStorage	Archivo <code>auth.test.js</code>
Objeto de usuario válido en formato JSON	Objeto mock serializado en localStorage	Archivos <code>auth.test.js</code> y <code>getToken.test.js</code>
Objeto de usuario inválido o JSON malformado	Cadenas de texto simuladas en localStorage	Archivo <code>auth.test.js</code>
Datos de sesión para limpiar autenticación	Registros simulados de token y usuario	Archivo <code>getToken.test.js</code> ; función <code>clearAuth()</code>
Respuestas exitosas de API	Mock de <code>fetch()</code>	Archivo <code>useFetchData.test.js</code> ; hook <code>useFetchData</code>
Respuestas fallidas de API	Mock de errores y rechazos de promesas	Archivo <code>useFetchData.test.js</code>
Datos para almacenamiento temporal en caché	Objetos, arreglos y valores simulados	Archivo <code>apiCache.test.js</code> ; utilidad <code>apiCache</code>
Datos de listas para paginación	Arreglos de elementos de prueba	Archivo <code>useListPagination.test.js</code>
Datos agregados para dashboard	Objetos mock de múltiples fuentes	Archivo <code>useDashboardData.test.js</code>
Entorno DOM para hooks y localStorage	jsdom	Directiva <code>@vitest-environment jsdom</code>
Entorno base de ejecución	Node.js	Configuración global de <code>vitest.config.js</code>
Token de autenticación válido	localStorage simulado mediante jsdom	Archivo <code>auth.test.js</code> ; módulo <code>auth.js</code>

TABLA 13. FUENTES DE DATOS PARA LAS PRUEBAS UNITARIAS (VALOR)

Las pruebas unitarias se apoyaron principalmente en datos simulados, almacenamiento local controlado, respuestas *mock* y entornos de ejecución configurados específicamente para pruebas automatizadas. Esta estrategia permitió validar unidades individuales sin depender de una base de datos productiva, servicios externos o disponibilidad de infraestructura institucional.

Para las pruebas de autenticación y almacenamiento se utilizó *localStorage* simulado mediante *jsdom*, lo cual permitió representar escenarios de token válido, token ausente, token expirado, usuario almacenado, usuario inexistente y datos corruptos. En el caso de *hooks* de datos, se emplearon respuestas simuladas de API para validar obtención de información, caché, errores de red y reintentos. Para las pruebas de paginación y *dashboard* se utilizaron arreglos y objetos de prueba que representaron listas, fuentes múltiples de datos y estructuras agregadas.

Requisito de dato para la prueba	Fuente	Uso dentro de la prueba
Token válido	localStorage simulado	Validar autenticación activa

Requisito de dato para la prueba	Fuente	Uso dentro de la prueba
Token inexistente	localStorage vacío	Validar sesión no iniciada
Token expirado	Mock de token con fecha vencida	Validar rechazo de sesión
Usuario autenticado	Objeto JSON simulado	Validar recuperación de usuario
Usuario corrupto	JSON inválido	Validar manejo seguro de error
Respuesta API exitosa	Mock de fetch	Validar carga de datos
Respuesta API fallida	Mock de error	Validar estado de error
Datos de caché	Objetos y arreglos simulados	Validar operaciones get, set y delete
Listas de elementos	Arreglos mock	Validar paginación
Datos de dashboard	Objetos agregados mock	Validar combinación de fuentes
Formulario de login	Entradas simuladas	Validar envío, errores y redirección
Entorno DOM	jsdom selectivo	Validar localStorage y componentes React
Entorno base	Node.js	Reducir consumo de memoria

TABLA 14. FUENTE DE DATOS PARA LAS PRUEBAS UNITARIAS (ARCHIVO)

4.1.4. Resultados de las pruebas

Los resultados de las pruebas unitarias fueron favorables en términos de estabilización del entorno, corrección de errores críticos y validación de componentes esenciales del *frontend*. Aunque la primera configuración provocó fallos de memoria, el proceso permitió identificar causas específicas, aplicar correcciones y transformar una suite inicialmente inestable en una configuración más ligera, ejecutable y adecuada para el contexto del proyecto.

El histórico de ejecución documentó veinte casos principales. De ellos, once fueron clasificados como aprobados, seis como listos para ejecución dentro de la suite activa, dos como resueltos o parciales y uno como pendiente de verificación final. La estadística general indicó que no permanecieron casos fallidos al cierre del proceso documentado, ya que el fallo inicial de memoria fue tratado mediante reconfiguración y exclusión de pruebas pesadas.

Uno de los resultados más importantes fue la corrección del manejo de errores en `useFetchData.js`. Al eliminar el relanzamiento de la excepción después del `catch`, se evitó la acumulación de promesas rechazadas no controladas. Esto contribuyó directamente a reducir fallos en cascada y mejorar la estabilidad general de la ejecución automatizada.

También se logró adaptar la configuración a **Vitest 4**, eliminando advertencias de configuración obsoleta y ajustando el modo de ejecución para mejorar la compatibilidad con **Windows**. El cambio hacia una estrategia mixta de entornos, con **Node.js** como base y `jsdom` solamente en archivos

necesarios, permitió reducir de manera significativa el consumo de memoria. El propio registro técnico reporta una reducción aproximada del 97% en la huella de memoria respecto a la configuración inicial.

Otro resultado relevante fue la reducción controlada de la suite de pruebas. Inicialmente existían más de cuatrocientas pruebas potenciales; sin embargo, su ejecución conjunta provocaba inestabilidad. Por ello, se conservaron aproximadamente entre 80 y 100 pruebas consideradas esenciales, distribuidas en archivos activos relacionados con autenticación, *tokens*, *hooks*, *cache*, *login*, *home* y pruebas simples de integración de página.

La validación de autenticación quedó cubierta mediante veintidós pruebas en `auth.test.js`, orientadas a comprobar recuperación de token, recuperación de usuario, validación de autenticación, expiración de token, cierre de sesión y ciclo completo de `login/logout`. Estas pruebas permitieron comprobar el comportamiento del sistema ante datos válidos, datos ausentes, *JSON* inválido, *tokens* malformados y eliminación de información de sesión.

La gestión de token y usuario también quedó validada mediante veintiún pruebas adicionales en `getToken.test.js`. Estas pruebas se enfocaron en operaciones de persistencia local, recuperación de datos y limpieza de autenticación. En conjunto con las pruebas de `auth.test.js`, se obtuvo una validación suficientemente amplia del comportamiento de sesión en el *frontend*.

Los *hooks* personalizados también fueron validados dentro de la suite activa. `useFetchData.test.js` incluyó once pruebas relacionadas con inicialización, obtención de datos, uso de *cache*, manejo de errores, reintentos e invalidación de *cache*. Por su parte, `useListPagination.test.js` incluyó diecinueve pruebas sobre navegación y cálculos de paginación, mientras que `useDashboardData.test.js` contempló ocho pruebas enfocadas en la combinación de múltiples fuentes de datos para la construcción del *dashboard*.

Además, se resolvieron errores estructurales en archivos de prueba. `HomePage.test.js`, que originalmente se encontraba vacío, fue corregido mediante la incorporación de una prueba mínima válida. También se creó el archivo `Login.test.js`, el cual no existía y generaba errores durante la ejecución. Aunque estas pruebas son básicas, su incorporación permitió mantener consistencia estructural dentro de la suite y evitar fallos innecesarios por archivos incompletos.

El resultado final de las pruebas unitarias fue satisfactorio. La suite activa quedó integrada por aproximadamente 107 pruebas automatizadas, con una tasa de éxito del 100%, tiempo de ejecución estimado entre 10 y 20 segundos, consumo de memoria menor a 1 GB y ausencia de bloqueos o errores críticos durante la ejecución.

La distribución de cobertura muestra que los módulos con mayor número de pruebas fueron autenticación, almacenamiento y paginación. Autenticación contó con 22 pruebas, almacenamiento con 21, paginación con 19, obtención de datos con 11, *cache API* con 8, *dashboard* con 8, integración de *login* con 5 y otros *hooks* o páginas con 13 pruebas. Esta distribución refleja una priorización adecuada sobre componentes esenciales para la operación del *frontend*.

También se documentó la exclusión temporal de 255 pruebas distribuidas en seis archivos: `useRegistro.test.js`, `useRegister.test.js`, `useOcr.test.js`, `formHelpers.test.js`, `validations.test.js` y `extras.test.js`. La decisión se tomó como medida de gestión de memoria, debido a que dichos archivos generaban demasiadas instancias *jsdom* o tenían una cantidad excesiva de pruebas para ejecutarse dentro de la suite activa sin afectar la estabilidad.

El ambiente de ejecución quedó configurado con *forks* como pool de ejecución secuencial, *Node.js* como ambiente predeterminado, *jsdom* selectivo para archivos que requieren DOM o *localStorage*, *timeout* de 60 segundos por prueba y aislamiento entre archivos. Esta configuración permitió mantener una ejecución estable, rápida y adecuada para el entorno de desarrollo del proyecto.

En conclusión, las pruebas unitarias permitieron validar funcionalidades críticas del *frontend*, incluyendo autenticación, almacenamiento, consumo de datos, caché, paginación, *dashboard* y *login*. La suite final quedó lista para ejecución y puede utilizarse como base para futuras iteraciones del sistema, ampliación de cobertura y validación previa a pruebas de sistema o pruebas de usabilidad.

5. Gestión de incidencias

La gestión de incidencias constituye una actividad fundamental dentro del proceso de aseguramiento de calidad del sistema “Tarjeta Más Juventud”, ya que permite registrar, clasificar, analizar y dar seguimiento a los errores o comportamientos no esperados detectados durante la ejecución de pruebas. Este proceso tiene como objetivo garantizar que las fallas identificadas sean documentadas adecuadamente y atendidas antes de la liberación del sistema.

Durante la ejecución de pruebas unitarias, de sistema y de usabilidad, las incidencias serán registradas mediante un control estructurado que incluirá información relacionada con el tipo de error, nivel de severidad, estado actual y responsable asignado para su corrección o seguimiento. La documentación de incidencias permitirá mantener trazabilidad sobre los problemas detectados, facilitando la priorización de actividades correctivas y el monitoreo del estado general de calidad del proyecto.

La clasificación de severidad será utilizada para determinar el impacto de cada incidencia dentro del funcionamiento del sistema. Las incidencias críticas corresponderán a errores que impidan la operación normal de funcionalidades esenciales, mientras que las incidencias medias o bajas estarán relacionadas con problemas visuales, inconsistencias menores o comportamientos que no comprometan directamente la operación principal del sistema.

Asimismo, cada incidencia podrá encontrarse en distintos estados, tales como “Pendiente”, “En proceso”, “Corregida”, “Validada” o “Cerrada”, dependiendo del avance de atención y validación correspondiente. Una vez aplicada la corrección, las pruebas relacionadas deberán ejecutarse nuevamente con el propósito de verificar que el problema haya sido resuelto satisfactoriamente y que no existan efectos secundarios derivados de la modificación implementada.

Las incidencias registradas durante las pruebas unitarias estuvieron principalmente relacionadas con problemas de configuración del entorno de pruebas, manejo de memoria, errores de asincronía, validaciones incorrectas y comportamiento inesperado en *hooks* personalizados. La siguiente tabla resume las principales incidencias identificadas durante el proceso de pruebas automatizadas del *frontend*.

ID	Incidencia	Severidad	Estado	Responsable
INC-001	Agotamiento de memoria durante la ejecución de más de 400 pruebas simultáneas en Vitest	Alta	Corregida	Equipo de desarrollo frontend
INC-002	Excepciones no controladas en useFetchData.js provocando acumulación de promesas rechazadas	Alta	Corregida	Equipo de desarrollo frontend
INC-003	Advertencias por configuración obsoleta de poolOptions.threads.singleThread en Vitest 4	Media	Corregida	Equipo de desarrollo frontend
INC-004	Error por referencia a window en entorno Node.js durante la ejecución de pruebas	Media	Corregida	Equipo de desarrollo frontend

ID	Incidencia	Severidad	Estado	Responsable
INC-005	Fallos de asincronía en hooks debido al uso incorrecto de act() y waitFor()	Alta	Corregida	Equipo de desarrollo frontend
INC-006	Variables NODE_OPTIONS no aplicadas correctamente en scripts npm sobre Windows	Media	Corregida	Equipo de desarrollo frontend
INC-007	localStorage indefinido durante pruebas ejecutadas en entorno Node.js	Alta	Corregida	Equipo de desarrollo frontend
INC-008	Consumo excesivo de memoria por uso global de jsdom en toda la suite de pruebas	Alta	Corregida	Equipo de desarrollo frontend
INC-009	Archivos de prueba con demasiadas instancias jsdom provocando inestabilidad	Media	Mitigada	Equipo de desarrollo frontend
INC-010	HomePage.test.js vacío generando error “No test suite found”	Baja	Corregida	Equipo de desarrollo frontend
INC-011	Archivo Login.test.js inexistente dentro de la suite activa de pruebas	Baja	Corregida	Equipo de desarrollo frontend
INC-012	Ejecución secuencial acumulando demasiado estado en memoria	Alta	Mitigada	Equipo de desarrollo frontend
INC-013	Hooks con alto número de renderizados afectando estabilidad general del entorno	Media	Mitigada	Equipo de desarrollo frontend
INC-014	Configuración inicial incompatible con el modelo de ejecución recomendado para Vitest 4.1.6	Media	Corregida	Equipo de desarrollo frontend
INC-015	Pruebas excluidas temporalmente debido a limitaciones de memoria y estabilidad	Baja	Pendiente de optimización	Equipo de desarrollo frontend
INC-012	Ejecución secuencial acumulando demasiado estado en memoria	Alta	Mitigada	Equipo de desarrollo frontend

TABLA 15. GESTIÓN DE INCIDENCIAS

6. Resultados y métricas

La evaluación de resultados y métricas permite medir el nivel de calidad alcanzado durante la ejecución de las pruebas realizadas sobre el sistema “Tarjeta Más Juventud”. Esta sección concentra información relacionada con la cantidad de pruebas ejecutadas, porcentaje de aprobación, incidencias detectadas y comportamiento general del entorno de pruebas, proporcionando una visión cuantitativa del estado actual de validación del sistema.

Las métricas obtenidas corresponden principalmente a la ejecución de pruebas unitarias automatizadas implementadas mediante *Vitest 4.1.6* y *React Testing Library* sobre el *frontend* del sistema. Los resultados reflejan el desempeño de la suite activa de pruebas después de las actividades de optimización, estabilización del entorno y corrección de incidencias técnicas relacionadas con consumo de memoria, configuración del *framework* y manejo de asincronía.

Asimismo, estas métricas permiten identificar el grado de cobertura funcional alcanzado en módulos críticos del sistema, tales como autenticación, almacenamiento local, consumo de datos, caché, paginación, *dashboard* y validaciones básicas de componentes de interfaz.

6.1. Casos ejecutados

Durante el proceso de validación se ejecutó una suite activa conformada por aproximadamente 107 pruebas automatizadas distribuidas en distintos módulos funcionales del *frontend*. Las pruebas fueron agrupadas en categorías relacionadas con autenticación, almacenamiento de sesión, *hooks* personalizados, obtención de datos, administración de caché, paginación y componentes básicos de interfaz.

La ejecución de la suite se realizó utilizando un entorno optimizado basado en Vitest 4.1.6, configurado con Node.js como ambiente predeterminado y *jsdom* selectivo para aquellos archivos que requerían funcionalidades del navegador, tales como *localStorage*, renderizado de componentes *React* y simulación del *DOM*. Esta configuración permitió ejecutar la totalidad de las pruebas activas dentro de un tiempo aproximado de 10 a 20 segundos y manteniendo un consumo de memoria inferior a 1 GB.

La distribución general de pruebas ejecutadas fue la siguiente:

Módulo	Total de pruebas ejecutadas
Autenticación	22
Almacenamiento	21
Obtención de datos	11
Caché API	8
Paginación	19
Dashboard	8
Integración Login	5
Otros hooks y páginas	13
Total general	107

TABLA 16. DISTRIBUCIÓN GENERAL DE LAS PRUEBAS EJECUTADAS

Los resultados obtenidos evidencian una priorización de pruebas sobre componentes críticos relacionados con autenticación, persistencia de información y manejo de datos dentro del *frontend* del sistema.

6.2. Casos aprobados

La totalidad de las pruebas activas ejecutadas finalizaron satisfactoriamente, alcanzando una tasa de éxito del 100% dentro de la suite considerada estable para el entorno de pruebas. Los resultados positivos indican que las funcionalidades validadas respondieron de acuerdo con el comportamiento esperado definido en los casos de prueba documentados.

Las pruebas aprobadas abarcaron distintos escenarios funcionales y técnicos, incluyendo:

- Recuperación y validación de tokens de autenticación.
- Persistencia y limpieza de información en `localStorage`.
- Obtención y procesamiento de datos desde servicios simulados.
- Almacenamiento y recuperación de datos en caché.
- Manejo de errores y recuperación ante fallos.
- Navegación y cálculos de paginación.
- Agregación de información para `dashboard`.
- Validaciones básicas del formulario de inicio de sesión.
- Configuración y estabilización del entorno *Vitest*.

Además, las actividades de corrección realizadas sobre *hooks*, configuración del *framework* y manejo de asincronía permitieron eliminar errores críticos detectados durante las primeras ejecuciones, dejando una suite estable y lista para reutilización en futuras iteraciones del proyecto.

La siguiente tabla resume el resultado general de aprobación:

Métrica	Resultado
Total de pruebas ejecutadas	107
Total de pruebas aprobadas	107
Porcentaje de éxito	100%
Errores críticos restantes	0
Crashes durante ejecución	0

TABLA 17. RESULTADO GENERAL DE APROBACIÓN

6.3. Casos fallidos

Durante las primeras etapas del proceso de pruebas se identificaron múltiples fallos relacionados principalmente con configuración del entorno, manejo de memoria y ejecución asíncrona de *hooks* personalizados. Sin embargo, dichas incidencias fueron corregidas progresivamente mediante actividades de diagnóstico, refactorización y optimización de la suite de pruebas.

Entre los problemas inicialmente detectados se encontraron agotamiento de memoria provocado por más de 400 pruebas simultáneas, uso excesivo de instancias `jsdom`, excepciones no controladas en *hooks* de datos, errores por referencias a objetos del navegador en entorno *Node.js* y configuraciones obsoletas incompatibles con *Vitest 4*.

Después de implementar las correcciones correspondientes, la suite activa quedó libre de fallos críticos, por lo que al cierre del proceso documentado no permanecieron casos fallidos dentro de las pruebas consideradas activas y estables.

No obstante, se decidió excluir temporalmente aproximadamente 255 pruebas pertenecientes a módulos con alto consumo de memoria y gran cantidad de renderizados. Esta decisión se tomó como medida preventiva para mantener la estabilidad general del entorno de pruebas y evitar nuevos bloqueos durante la ejecución secuencial.

La siguiente tabla resume el estado final de fallos documentados:

Métrica	Resultado
Casos fallidos activos	0
Casos corregidos durante el proceso	15
Pruebas excluidas temporalmente	255
Motivo principal de exclusión	Consumo excesivo de memoria
Estado final de la suite activa	Estable

TABLA 18. ESTADO FINAL DE LOS CASOS FALLIDOS

6.4. Incidencias

Durante la ejecución de las pruebas unitarias se registraron distintas incidencias técnicas relacionadas con estabilidad, compatibilidad y optimización del entorno de pruebas. Las incidencias fueron documentadas, clasificadas según severidad y posteriormente atendidas mediante actividades correctivas implementadas sobre archivos de configuración, *hooks* personalizados y scripts auxiliares del entorno.

Las incidencias más relevantes estuvieron relacionadas con:

- Agotamiento de memoria durante ejecución masiva de pruebas.
- Excepciones no controladas en *hooks* asíncronos.
- Configuraciones incompatibles con **Vitest 4.1.6**.
- Uso incorrecto de `act()` y `waitFor()` en pruebas **React**.
- Errores por ausencia de `window` y `localStorage` en entorno **Node.js**.
- Archivos de prueba inexistentes o vacíos.
- Exceso de instancias `jsdom` durante ejecución secuencial.

Como resultado del proceso de corrección y estabilización, las incidencias críticas fueron resueltas exitosamente y el entorno quedó configurado para ejecutar las pruebas activas sin interrupciones, bloqueos o errores de memoria.

La siguiente tabla resume las métricas generales de incidencias:

Métrica de incidencias	Resultado
Total de incidencias registradas	15
Incidencias críticas	5
Incidencias medias	7
Incidencias bajas	3

Incidencias corregidas	14
Incidencias mitigadas	1
Incidencias pendientes críticas	0

TABLA 19. MÉTRICAS GENERALES DE LAS INCIDENCIAS

Apéndice A: Evidencias

The screenshot displays the Vitest test runner interface in a web browser. The top section shows the test results summary: 392 Pass, 0 Fail, and 392 Total. Below this, a list of test files and their results is shown, including:

- src/hooks/_tests_/useFetchData.test.js (90ms)
- src/hooks/_tests_/useFetchData.test.js (580ms)
- src/hooks/_tests_/useFetchData.test.js (74ms)
- src/hooks/_tests_/useFetchData.test.js (87ms)
- src/hooks/_tests_/useFetchData.test.js (17ms)
- src/hooks/_tests_/useFetchData.test.js (92ms)
- src/hooks/_tests_/useFetchData.test.js (7ms)
- src/hooks/_tests_/useFetchData.test.js (7ms)
- src/hooks/_tests_/useFetchData.test.js (95ms)
- src/hooks/_tests_/useFetchData.test.js (8ms)
- src/hooks/_tests_/useFetchData.test.js (77ms)
- src/hooks/_tests_/useFetchData.test.js (8ms)
- src/hooks/_tests_/useFetchData.test.js (17ms)
- src/hooks/_tests_/useListPagination.test.js (171ms)
- src/hooks/_tests_/useListPagination.test.js (80ms)
- src/hooks/_tests_/useListPagination.test.js (54ms)
- src/hooks/_tests_/useListPagination.test.js (14ms)
- src/hooks/_tests_/useListPagination.test.js (10ms)
- src/hooks/_tests_/useListPagination.test.js (85ms)

The bottom section shows a terminal window with the command `npm run test:ui` and the output of the test runner, including the same summary as above and the command `press h to show help, press q to quit`.